المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⵉⵎⵙⵙⵓⵜ ⵜⵏⵎⵓⵔⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵓⵏⵉⵏ ⵜⵉⵏⵏⵓⵔⵉⵏ - ⴰⴽⵜ ⵎⵍⵍⴰⵍ
Ecole Nationale des Sciences Appliquées - Béni Mellal

Filière : **Intelligence Artificielle et Cybersécurité**

Module : **Blockchain**

Plateforme de financement participatif décentralisée à gouvernance par jeton

Rapport de mini-projet · novembre 2026

Réalisé par

Ferdaous Kouzi
Hassnae Latouti

Encadré par

Pr. Hamza Touil

Année universitaire : 2026–2027

Table des matières

1 Introduction et cahier des charges 3
1.1 Contexte 3
1.2 Exigences fonctionnelles 3
1.3 Exigences de sécurité 3
2 Architecture 4
2.1 Vue d'ensemble 4
2.2 Flux principaux 4
2.3 Statuts d'une campagne 5
2.4 Métadonnées sur IPFS 5
3 Choix techniques 6
3.1 Hardhat 3 6
3.2 OpenZeppelin 6
3.3 Paramètres de conception 6
3.4 Retrait et gouvernance 6
4 Implémentation des contrats 7
4.1 Jeton CFD (M1) 7
4.2 Cœur du financement (M2) 7
4.3 Trésorerie multisignature 7
4.4 Gouvernance (M3) 7
5 Analyse de sécurité 8
5.1 Anti-réentrance 8
5.2 Contrôle d'accès 8
5.3 Circuit breaker et multisignature 8
5.4 Anti-manipulation de la gouvernance 9
6 Tests 10
6.1 Tests unitaires et d'intégration 10
6.2 Fuzzing 10
7 Audit statique avec Slither 11
8 Interface web (M4) 12
9 Déploiement 14
9.1 Chaîne locale 14
9.2 Réseau de test Sepolia 14
10 Limites connues 17
11 Perspectives 18
12 Conclusion 19
Annexes 20

1 Introduction et cahier des charges

1.1 Contexte

Ce projet consiste à réaliser une application décentralisée (dApp) complète de financement participatif, du contrat intelligent jusqu'à l'interface utilisateur, sans intermédiaire centralisé. Les porteurs de projet créent des campagnes. Les contributeurs financent en ETH et reçoivent en retour un jeton de gouvernance ERC-20 nommé **CFD**, proportionnel à leur apport. Les décisions sensibles (déblocage anticipé de fonds, gel d'une campagne suspecte) sont tranchées par un vote pondéré par la détention de jetons, et non par un administrateur unique.

Le projet est évalué comme s'il gérait de vrais fonds : la sécurité est donc au centre de la conception, autour de trois familles de vulnérabilités étudiées en cours (réentrance, contrôle d'accès, manipulation de gouvernance).

1.2 Exigences fonctionnelles

Module	Exigence	Réalisation
M1	Jeton ERC-20 « CFD » via OpenZeppelin, frappe automatique à chaque contribution	`CFDToken`
M2	Campagnes, contributions, quatre statuts, remboursement, retrait réservé au bénéficiaire	`CrowdfundingCore`, `MultiSigTreasury`
M3	Vote pondéré par les CFD, quorum, majorité, délai avant exécution automatique	`Governance`
M4	Interface connectée à MetaMask via Ethers.js : campagnes, contribution, vote, trésorerie	Front-end React, Vite, TypeScript

1.3 Exigences de sécurité

Le sujet impose : protection anti-réentrance, contrôle d'accès strict, circuit breaker, multisignature 2 sur 3 pour la trésorerie, pouvoir de vote calculé sur un solde figé (snapshot), métadonnées sur IPFS (seul le hash est enregistré dans la chaîne), tests unitaires avec au moins un test de fuzzing, et audit statique avec Slither ou Mythril. La section 5 montre comment chacune est traitée.

2 Architecture

2.1 Vue d'ensemble

Le système comporte quatre contrats Solidity qui communiquent par des rôles `AccessControl` :

- **CFDToken** : jeton ERC-20 avec l'extension `ERC20Votes`, qui enregistre les soldes bloc par bloc (snapshots).
- **CrowdfundingCore** : gère les campagnes et l'argent. C'est le seul contrat autorisé à frapper des CFD (`MINTER_ROLE`).
- **MultiSigTreasury** : trésorerie à trois propriétaires, qui reçoit les frais de plateforme.
- **Governance** : vote des détenteurs de CFD. Elle détient le `GOVERNANCE_ROLE` du Core et exécute le résultat des votes.

Deux interfaces, `ICFDToken` et `ICrowdfunding`, découplent les contrats. Elles ont permis de tester la gouvernance avec un faux Core avant la fusion des deux parties.

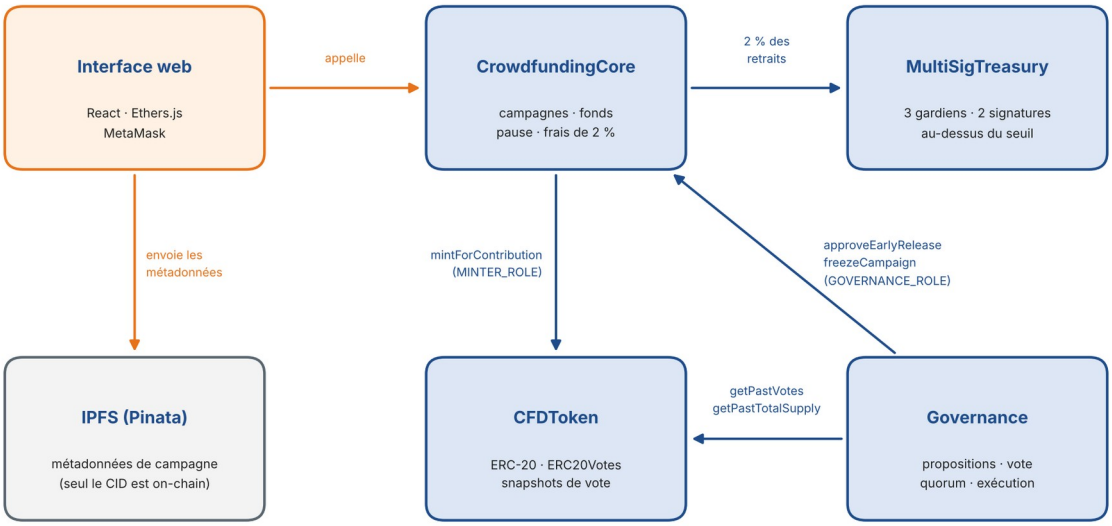


Figure 1 : Architecture des contrats et des rôles. L'interface appelle aussi Governance et la trésorerie (flèches omises pour la lisibilité).

2.2 Flux principaux

1. Un porteur de projet appelle `createCampaign` avec un objectif, une durée et le hash IPFS des métadonnées.
2. Un contributeur appelle `contribute(id)` en envoyant de l'ETH. Le Core met à jour ses comptes, puis demande au jeton de frapper les CFD (1 ETH pour 1000 CFD) et de déléguer le vote au contributeur.
3. Si l'objectif n'est pas atteint à l'échéance, chaque contributeur appelle `refund`. Si la campagne est gelée, le remboursement est également possible.
4. Si l'objectif est atteint, le bénéficiaire appelle `withdrawFunds`. Le Core verse 98 % au bénéficiaire et 2 % à la trésorerie.
5. Un détenteur de CFD crée une proposition de déblocage anticipé ou de gel. Les détenteurs votent. Après le délai de vote, la proposition est exécutée et appelle le Core.

2.3 Statuts d'une campagne

Le statut est calculé à la demande par `getStatus`, selon cet ordre de priorité :

Statut	Condition
Annulée (*Cancelled*)	la campagne a été gelée par la gouvernance
Réussie (*Successful*)	le montant levé atteint l'objectif
En cours (*Active*)	l'échéance n'est pas passée
Échouée (*Failed*)	l'échéance est passée sans atteindre l'objectif

Le remboursement est possible pour les statuts *Failed* et *Cancelled*.

2.4 Métadonnées sur IPFS

Le titre et la description d'une campagne sont envoyés à IPFS par le service Pinata au moment de la création. Seul le identifiant du contenu (CID) est enregistré dans le contrat (`ipfsHash`), ce qui limite le coût en gaz : la chaîne garde l'empreinte, IPFS garde les données. À l'affichage, l'interface relit le contenu à partir du CID. Le contrat refuse un hash vide dans `createCampaign`.

3 Choix techniques

3.1 Hardhat 3

Nous avons choisi Hardhat 3 pour trois raisons. Il permet d'écrire des tests de fuzzing en Solidity sans outil supplémentaire. Il conserve la compatibilité avec des tests Mocha en TypeScript et Ethers.js, bibliothèque imposée pour le front-end, ce qui donne le même langage de bout en bout. Enfin, il compile les contrats et produit les ABI utilisés par l'interface. Son inconvénient est de disposer de moins de documentation que la version 2.

3.2 OpenZeppelin

Nous réutilisons des composants audités plutôt que de réécrire des briques sensibles : `ERC20`, `ERC20Permit`, `ERC20Votes`, `AccessControl`, `Pausable` et `ReentrancyGuard` (version 5). La logique métier (campagnes, frais, statuts, trésorerie multisignature, gouvernance) est écrite par l'équipe.

3.3 Paramètres de conception

Le sujet demande de définir et justifier le taux, le quorum et la majorité.

Paramètre	Valeur	Justification
Taux de frappe	1 ETH = 1000 CFD	Entier simple, proportionnel à l'apport comme l'exige le sujet. Avec 18 décimales, même une très petite contribution donne un pouvoir de vote non nul.
Frais de plateforme	2 % du retrait	Finance la plateforme sans intermédiaire ; envoyés à la trésorerie multisignature. La division entière n'égare aucun wei : frais plus versement valent exactement le total (vérifié par fuzzing).
Quorum	10 % de l'offre totale, fixé à la création	Atteignable avec une participation modérée, sans qu'un petit détenteur décide seul. Fixé à la création, il ne peut pas être manipulé pendant le vote.
Majorité	Simple (pour > contre)	Règle la plus lisible. Un seuil plus élevé bloquerait trop facilement les décisions urgentes comme le gel d'une campagne suspecte ; le quorum écarte les décisions prises par trop peu de votants.
Durée du vote	3 jours	Laisse aux contributeurs le temps de réagir sans retarder inutilement un gel. Un script avance l'horloge locale de 3 jours pour tester l'exécution.
Seuil du multisig	1 ETH en local, 0,01 ETH sur Sepolia	Au-dessus du seuil, deux signatures sur trois sont obligatoires. Le seuil est abaissé sur le testnet pour rendre la règle visible en démonstration.
Délégation du vote	Automatique à la première réception de CFD	`ERC20Votes` ne compte aucun poids sans délégation. La frappe délègue d'office, sans écraser une délégation déjà choisie.

3.4 Retrait et gouvernance

Le sujet conditionne `withdrawFunds()` à la gouvernance. Notre lecture : un gel voté bloque le retrait et ouvre le remboursement ; avant l'échéance, le retrait exige un déblocage voté ; après l'échéance, une campagne réussie et non gelée retire sans vote, pour que la majorité n'ait pas un droit de blocage sur chaque projet réussi. Limite assumée : le gel doit intervenir pendant la collecte.

4 Implémentation des contrats

4.1 Jeton CFD (M1)

`CFDToken` hérite de `ERC20`, `ERC20Permit`, `ERC20Votes` et `AccessControl`. La fonction `mintForContribution(to, weiAmount)` est réservée au `MINTER_ROLE`, que seul le Core possède. Elle frappe `weiAmount` multiplié par 1000 jetons et renvoie le montant frappé. Avec `ERC20Votes`, détenir des jetons ne donne aucun pouvoir de vote tant que l'adresse ne s'est pas délégué ses votes : la fonction de frappe délègue donc automatiquement le vote au bénéficiaire lors de sa première réception, sans écraser une délégation déjà choisie.

4.2 Cœur du financement (M2)

`CrowdfundingCore` hérite de `AccessControl`, `Pausable` et `ReentrancyGuard`. Il implémente la création de campagne, la contribution, le remboursement, le retrait et les fonctions réservées à la gouvernance (`freezeCampaign`, `approveEarlyRelease`). Une campagne enregistre son bénéficiaire, son objectif, son échéance, le montant levé, le hash IPFS et trois indicateurs (gelée, déblocage anticipé approuvé, retirée). Le Core vérifie la valeur renvoyée par `mintForContribution`. Le retrait est possible pour le bénéficiaire lorsque l'objectif est atteint et que l'échéance est passée, ou lorsque la gouvernance a approuvé un déblocage anticipé.

4.3 Trésorerie multisignature

`MultiSigTreasury` a exactement trois propriétaires. Une transaction est soumise par un propriétaire, qui la confirme d'office, puis confirmée par les autres. Au-dessus d'une limite définie au déploiement (`smallLimit`), deux confirmations sur trois sont obligatoires. Un propriétaire peut révoquer sa confirmation avant l'exécution.

4.4 Gouvernance (M3)

`Governance` permet de créer une proposition (déblocage anticipé ou gel d'une campagne), de voter pour ou contre, puis de l'exécuter une fois le délai écoulé. `propose` refuse une campagne inexistante (`campaignCount()`) et exige un pouvoir de vote au snapshot. Le poids d'un vote est lu sur le solde figé au bloc précédant la création de la proposition, grâce à `getPastVotes` ; chaque adresse ne vote qu'une fois. Le quorum est calculé et fixé à la création. À l'exécution, le contrat marque la proposition comme exécutée *avant* d'appeler le Core, ce qui bloque la double exécution et la réentrance. Le Core n'obéit qu'au contrat `Governance` : même son administrateur ne peut pas contourner le vote.

5 Analyse de sécurité

Chaque exigence de sécurité est reliée à une menace, à une notion du cours, à son implémentation et à une preuve.

Exigence	Menace	Notion de cours	Implémentation	Preuve
Anti-réentrance	Un contrat malveillant rappelle `refund` ou `withdrawFunds` pendant le transfert d'ETH (attaque de The DAO)	Checks-Effects-Interactions, mutex	État mis à jour avant l'envoi d'ETH ; `nonReentrant`	`Reentrancy.ts` simule l'attaque et prouve son échec
Contrôle d'accès	Un tiers appelle une fonction d'administration	Moindre privilège, rôles	`AccessControl`, rôles de la section 5.2	Tests d'accès refusé
Circuit breaker	Un incident exige d'arrêter la plateforme	Disjoncteur d'urgence	`Pausable`, `pause()` réservée à `PAUSER_ROLE`	Tests de pause et de reprise
Multisig 2 sur 3	Une clé compromise vide la trésorerie	Séparation des pouvoirs	`MultiSigTreasury`	22 tests ; écran dans l'interface
Anti-manipulation du vote	Acheter des jetons juste avant un vote, voter, les revendre	Snapshot	Poids lu au bloc `block.number - 1` ; quorum fixé à la création	Test dédié dans `Governance.ts`

5.1 Anti-réentrance

Menace. Un contrat malveillant dont la fonction `receive` se déclenche à la réception d'ETH rappelle `refund` ou `withdrawFunds` avant la fin du premier appel, et vide le contrat (attaque de The DAO, 2016).

Défenses indépendantes. Le modificateur `nonReentrant` protège `contribute`, `refund` et `withdrawFunds`. Par ailleurs, le motif Checks-Effects-Interactions met à jour l'état avant tout envoi d'ETH : le solde du contributeur est mis à zéro avant le remboursement, et l'indicateur de retrait est activé avant les envois.

Preuve. `Reentrancy.ts` déploie un contrat pirate (`ReentrancyAttacker.sol`) qui rappelle `refund` puis `withdrawFunds` depuis `receive()`. Les deux tests passent : la seconde tentative est bloquée et les fonds des autres contributeurs restent intacts.

5.2 Contrôle d'accès

Rôle	Contrat	Détenteur	Fonctions protégées
`MINTER_ROLE`	`CFDToken`	Contrat Core uniquement	frappe de CFD
`GOVERNANCE_ROLE`	`CrowdfundingCore`	Contrat Governance uniquement	gel, déblocage anticipé
`PAUSER_ROLE`	`CrowdfundingCore`	Administrateur	pause et reprise
`DEFAULT_ADMIN_ROLE`	Jeton et Core	Administrateur	attribution des rôles

Les tests vérifient qu'un compte sans rôle est refusé pour chacune de ces fonctions. `approveEarlyRelease` et `freezeCampaign` ne sont appelables que par `Governance` : même l'administrateur du Core ne peut pas contourner le vote.

5.3 Circuit breaker et multisignature

Le Core hérite de `Pausable` : un compte ayant le `PAUSER_ROLE` suspend la création de campagne, les contributions, les remboursements et les retraits en cas d'incident, puis les rouvre. Pour la trésorerie, deux

confirmations sur trois sont exigées au-dessus du seuil ; les 22 tests couvrent le déploiement, la double confirmation, la révocation, la double exécution et le manque de fonds.

5.4 Anti-manipulation de la gouvernance

Menace. Un attaquant achète ou emprunte massivement des jetons juste avant un vote, vote, puis les revend (prise de contrôle éclair).

Défense. Le pouvoir de vote est lu sur un solde figé au bloc précédant la création de la proposition. Acheter des jetons après la création de la proposition ne donne donc aucun poids supplémentaire, et le quorum est fixé à la création pour la même raison. Un test dédié le vérifie (jetons achetés après la proposition ignorés), ainsi que les tests du jeton sur `getPastVotes` et `getPastTotalSupply`.

Remarque. Le poids de vote provient des contributions. Une limite subsiste : un contributeur conserve ses CFD après un remboursement (section 10), ce qui dissocie en partie le poids de vote d'un engagement financier durable. Brûler les CFD au remboursement est la correction identifiée.

6 Tests

6.1 Tests unitaires et d'intégration

Suite	Tests	État
`CFDToken.ts`	14	réussis
`CrowdfundingCore.ts`	23	réussis
`MultiSigTreasury.ts`	22	réussis
`Reentrancy.ts`	2	réussis
`Governance.ts` (avec mocks)	13	réussis
`Integration.ts` (vrais contrats)	4	réussis
`CrowdfundingFuzz.t.sol` (fuzzing)	3	réussis
Total	**81**	**réussis (78 Mocha, 3 Solidity)**

La gouvernance a d'abord été testée contre un faux Core (`MockCore`) grâce à l'interface `ICrowdfunding`. `Integration.ts` confirme ensuite le comportement avec les vrais contrats.

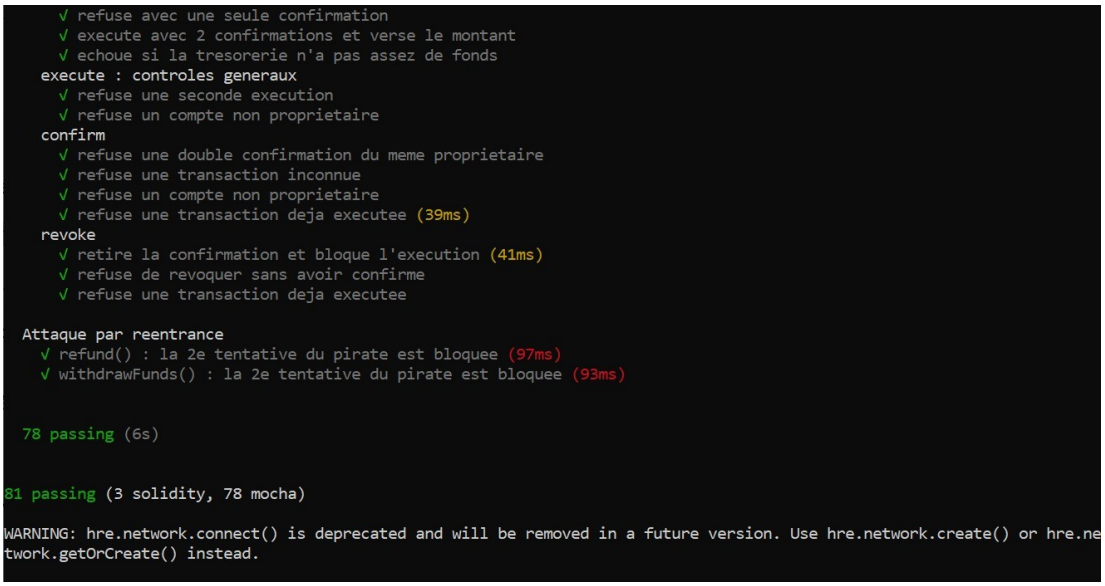


Figure 2 : Fin de la sortie de `npx hardhat test` : 81 tests réussis (3 Solidity, 78 Mocha).

6.2 Fuzzing

Trois tests de fuzzing, écrits en Solidity, vérifient pour 256 montants aléatoires chacun des propriétés qui doivent toujours rester vraies :

1. une contribution de x wei frappe exactement 1000 × x unités de CFD ;
2. un remboursement après échec rend exactement la contribution et laisse le contrat vide ;
3. au retrait, frais plus versement égalent exactement le total : aucun wei n'est perdu, même pour de très petits montants.

7 Audit statique avec Slither

Outil : Slither 0.11.6, lancé sur le projet complet, `Governance` incluse, avec exclusion des bibliothèques, des mocks et des tests (`slither . --filter-paths "node_modules|test|mocks|Fuzz"`). Sans ce filtre, l'outil remonte 606 résultats, presque tous dans les bibliothèques. **Résultat final : 12 résultats informatifs, aucune vulnérabilité de niveau haut ou moyen.**

Étape	Résultats
Passage 1 (jeton, Core, trésorerie)	11
Après correction de `missing-inheritance`	11 (apparition de `unused-return`)
Après correction de `unused-return`	10
Projet complet avec `Governance`	12 (9 `timestamp`, 3 `low-level-calls`)

Corrections. `CFDToken` hérite désormais explicitement de `ICFDToken` (déplacée dans un fichier propre), et le Core vérifie la valeur renvoyée par la fonction de frappe, qu'il ignorait. La seconde correction est née de la première : l'outil a été relancé après chaque modification. L'ajout de `Governance` fait passer les alertes `timestamp` de 7 à 9.

Alertes acceptées et justifiées.

Détecteur	Nombre	Justification
`timestamp`	9	Les échéances durent des jours : une dérive d'environ 15 secondes d'un validateur est sans effet. Le poids de vote est figé au bloc, pas au temps.
`low-level-calls`	3	`call{value}` est la méthode recommandée pour envoyer de l'ETH. Le retour est vérifié, `nonReentrant` et Checks-Effects-Interactions sont en place, et `Reentrancy.ts` le prouve.

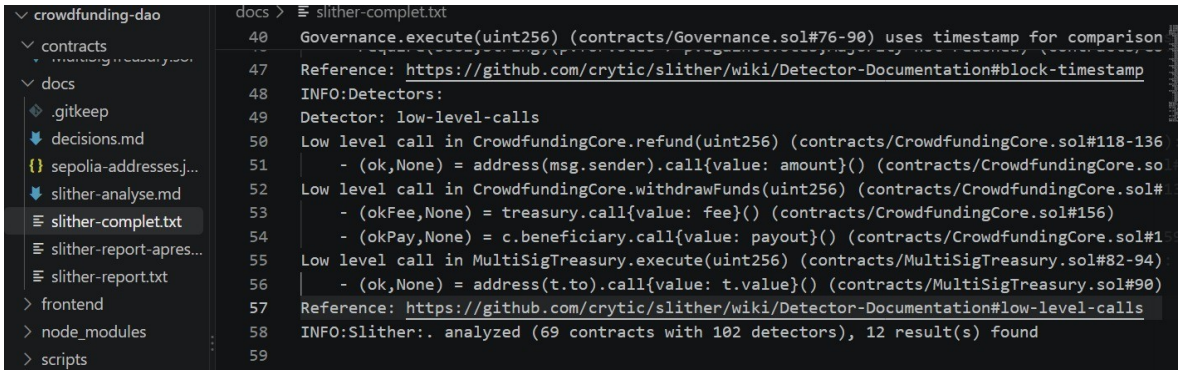


Figure 3 : Fin de la sortie Slither sur le projet complet : 12 résultats (fichier `docs/slither-complet.txt`).

8 Interface web (M4)

L'interface est développée avec Vite, React, TypeScript et Ethers.js. Elle se connecte à MetaMask et comprend :

- une page d'accueil avec des cartes de synthèse lues sur la chaîne (campagnes, dernier vote, fonds détenus par le Core) ;
- **Campagnes** : création avec métadonnées IPFS, contribution, remboursement, retrait ;
- **Gouvernance** : création d'une proposition, vote pour ou contre, statuts (adoptée, rejetée, exécutée), exécution ;
- **Coffre multisig** : gardiens, solde, dépôt, proposition d'un paiement, signature, retrait de signature, exécution.

Les captures ci-dessous sont prises en réseau local Hardhat (chaîne 31337) avec MetaMask.

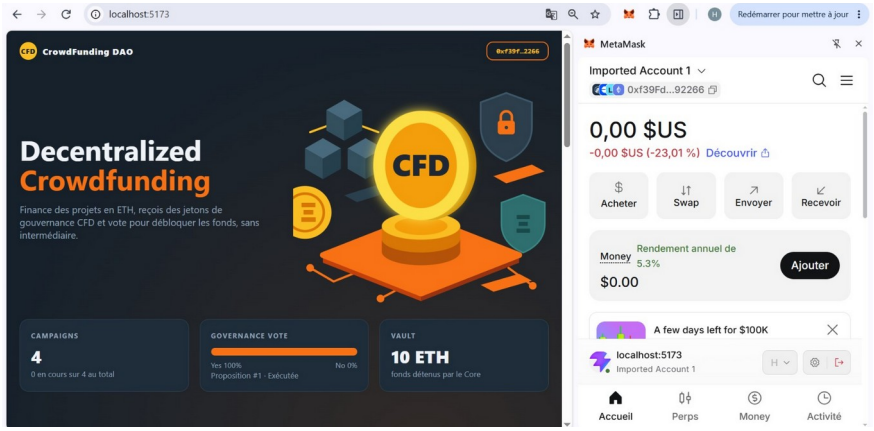


Figure 4 : Page d'accueil connectée à MetaMask : cartes de synthèse lues sur la chaîne.

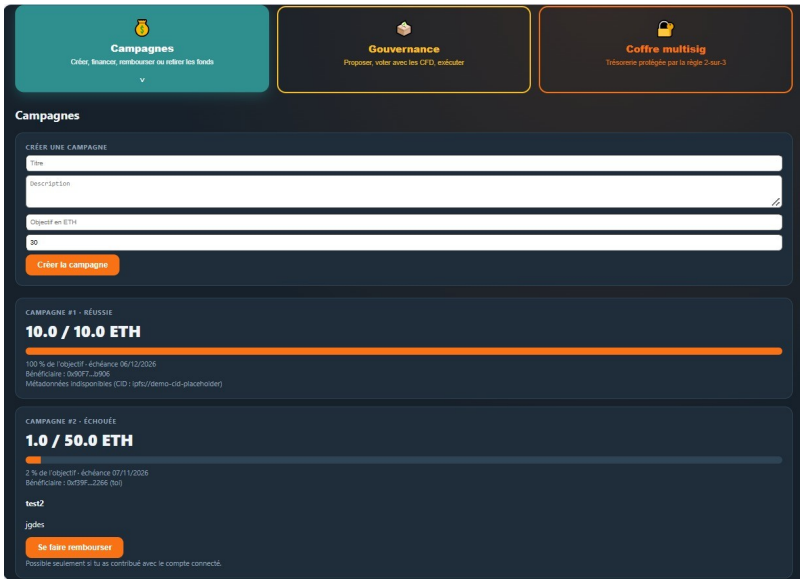


Figure 5 : Onglet Campagnes : formulaire de création, campagne réussie, campagne échouée avec remboursement ; titre et description de la campagne n°2 relus sur IPFS.

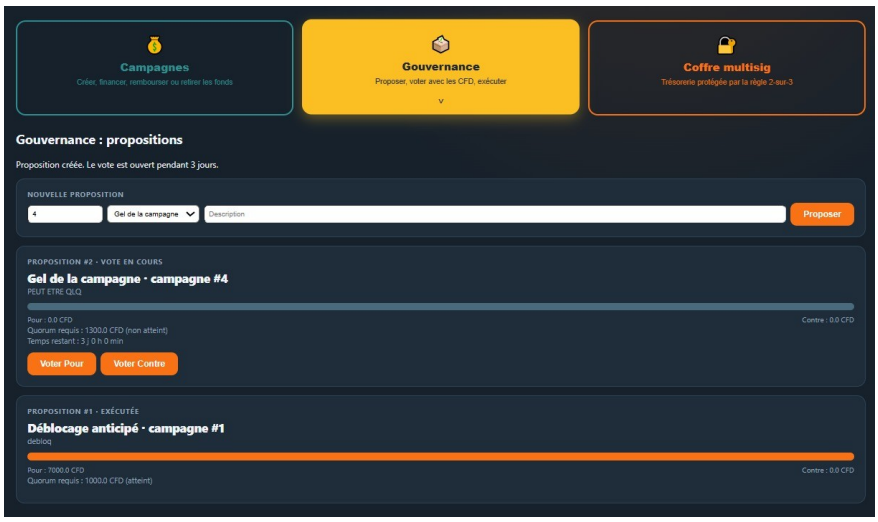


Figure 6 : *Onglet Gouvernance : un gel en cours de vote, et un déblocage anticipé exécuté (7000 CFD pour, quorum atteint).*

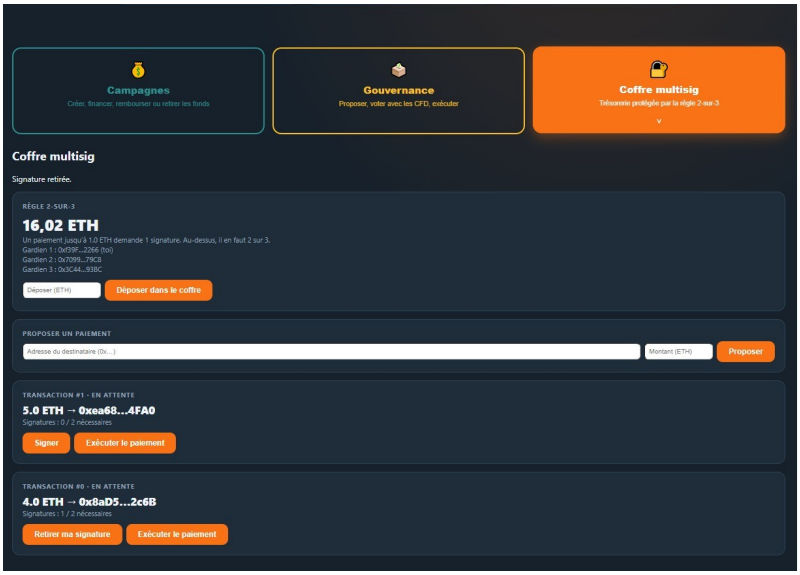


Figure 7 : *Onglet Coffre multisig : règle 2 sur 3, transactions en attente avec 0 sur 2 et 1 sur 2 signatures.*

9 Déploiement

9.1 Chaîne locale

Un script de déploiement installe les quatre contrats, attribue le `MINTER_ROLE` au Core et le `GOVERNANCE_ROLE` à `Governance`, puis crée des campagnes et des contributions de démonstration. Un second script avance l'horloge de la chaîne locale de trois jours pour tester l'exécution d'une proposition.

La figure 8 montre la sortie du script sur la chaîne locale : adresses des quatre contrats, campagne de démonstration à 6 ETH collectés, et CFD détenus par les trois premiers comptes (1 ETH contribué = 1000 CFD).



Figure 8 : Déploiement local (hardhat run scripts/deploy.ts --network localhost) : contrats, campagne 1 et soldes CFD.

9.2 Réseau de test Sepolia

Les contrats sont déployés et **vérifiés sur Etherscan et Sourcify** (chaîne 11155111) par un compte de test dédié. Les secrets sont dans un fichier `.env` ignoré par Git. Les rôles `MINTER_ROLE` (jeton vers Core) et `GOVERNANCE_ROLE` (Core vers `Governance`) sont attribués par le script de déploiement. Le seuil du multisig est de 0,01 ETH sur ce réseau.

Contrat	Adresse Sepolia
`CFDToken`	`0xEf59E048c1944e8760048fAc361758fA028A60A3`
`MultiSigTreasury`	`0x50A40978c8E5BaF1BEBAE7802E98747b30F0a841`
`CrowdfundingCore`	`0x341605Eb699b90A3c2B2e3c564C674EB60d145e2`
`Governance`	`0x37B3A16669ec772ED6B62E8EbE90955F5Ca9E265`

Les adresses sont aussi enregistrées dans `docs/sepolia-addresses.json`.

Le compte déployeur a d'abord été approvisionné en ETH de test par un faucet Sepolia (figure 9). Le script check-sepolia vérifie ensuite la configuration, puis deploy-sepolia déploie les quatre contrats et attribue les deux rôles (figure 10).

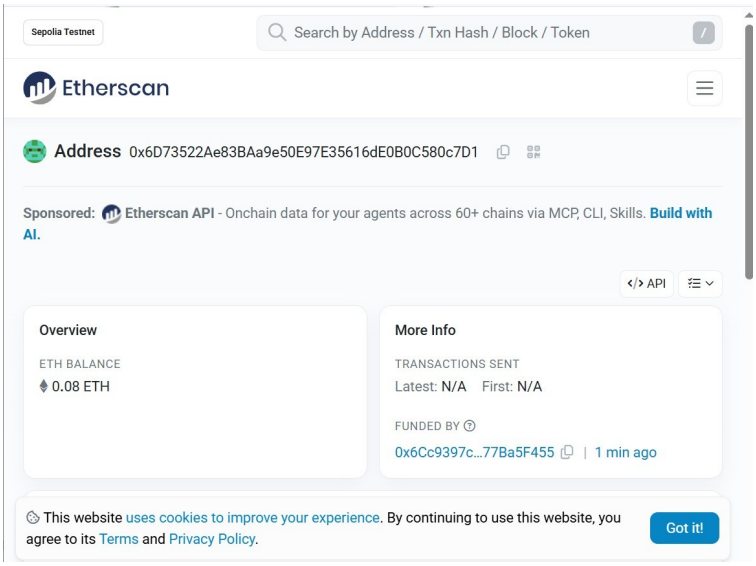


Figure 9 : Compte déployeur sur Etherscan (Sepolia) : solde de 0,08 ETH de test après réception du faucet.

```
C:\Users\LATOUTI\miniProjetBlck\crowdfunding-dao>npx hardhat run scripts/deploy
-sepolia.ts --network sepolia

Déployeur : 0x6D73522Ae83BAa9e50E97E35616dE0B0C580c7D1
Solde     : 0.08 ETH
CFDToken          : 0xEf59E048c1944e8760048fAc361758fA028A60A3
MultiSigTreasury : 0x50A40978c8E5BaF1BEBAE7802E98747b30F0a841
CrowdfundingCore : 0x341605Eb699b90A3c2B2e3c564C674EB60d145e2
Governance        : 0x37B3A16669ec772ED6B62E8EbE90955F5Ca9E265
MINTER_ROLE donné au Core
GOVERNANCE_ROLE donné à Governance
Adresses enregistrées dans docs/sepolia-addresses.json
Solde restant : 0.00980971 ETH
WARNING: hre.network.connect() is deprecated and will be removed in a future ve
rsion. Use hre.network.create() or hre.network.getOrCreate() instead.
```

Figure 10 : Déploiement sur Sepolia : adresses des quatre contrats, attribution de MINTER_ROLE au Core et de GOVERNANCE_ROLE à Governance.

La vérification du code source est faite sur Sourcify, qui compare le bytecode déployé aux sources publiées (figures 11 et 12). Les pages Etherscan de Governance et de CrowdfundingCore confirment la présence des contrats à l'adresse déployée, créés par le compte déployeur (figures 13 et 14).

```
  contracts/CFDToken.sol:CFDToken
  Address: 0xEf59E048c1944e8760048fAc361758fA028A60A3

⏳ Waiting for verification result...


The initial verification attempt for contracts/CFDToken.sol:CFDToken failed usi
ng the minimal compiler input.

Trying again with the full solc input used to compile and deploy the contract.
Unrelated contracts may be displayed on Sourcify as a result.


⏳  Submitted source code for verification on Sourcify:

  contracts/CFDToken.sol:CFDToken
  Address: 0xEf59E048c1944e8760048fAc361758fA028A60A3

⏳ Waiting for verification result...


⏳ Contract verified successfully on Sourcify!

  contracts/CFDToken.sol:CFDToken
  Explorer: https://sourcify.dev/server/repo-ui/11155111/0xEf59E048c1944e876004
8fAc361758fA028A60A3
```

Figure 11 : Vérification de CFDToken sur Sourcify (chaîne 11155111) : « Contract verified successfully ».

```
  contracts/MultiSigTreasury.sol:MultiSigTreasury
  Address: 0x50A40978c8E5BaF1BEBAE7802E98747b30F0a841

 Waiting for verification result...


The initial verification attempt for contracts/MultiSigTreasury.sol:MultiSigTre
asury failed using the minimal compiler input.

Trying again with the full solc input used to compile and deploy the contract.
Unrelated contracts may be displayed on Sourcify as a result.


  Submitted source code for verification on Sourcify:

  contracts/MultiSigTreasury.sol:MultiSigTreasury
  Address: 0x50A40978c8E5BaF1BEBAE7802E98747b30F0a841

 Waiting for verification result...


 Contract verified successfully on Sourcify!

  contracts/MultiSigTreasury.sol:MultiSigTreasury
  Explorer: https://sourcify.dev/server/repo-ui/11155111/0x50A40978c8E5BaF1BEBA
E7802E98747b30F0a841
```

Figure 12 : *Vérification de MultiSigTreasury sur Sourcify (chaîne 11155111).*

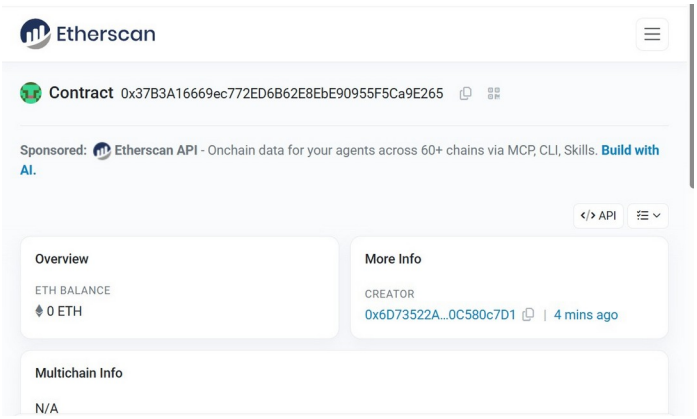


Figure 13 : *Contrat Governance sur Etherscan Sepolia : créateur = compte déployeur.*

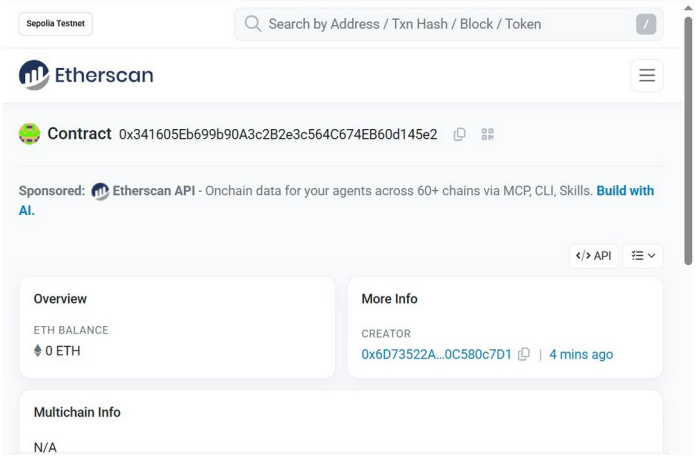


Figure 14 : *Contrat CrowdfundingCore sur Etherscan Sepolia : créateur = compte déployeur.*

10 Limites connues

Limite	Piste
Un contributeur garde ses CFD après un remboursement : il conserve du pouvoir de vote sans engagement financier durable	Brûler les CFD au remboursement
Sous `smallLimit`, une seule signature suffit : un gros retrait peut être découpé	Plafond cumulé par période
Le retrait normal ne passe pas par un vote, et le gel doit intervenir pendant la collecte	Ajouter un type de proposition de retrait
Le bénéficiaire peut contribuer à sa propre campagne	L'exclure des contributeurs
Le JWT Pinata est lu par le front, donc visible dans le navigateur	Petit serveur intermédiaire ; clé limitée à l'envoi de fichiers
Les trois gardiens du multisig sont contrôlés par l'équipe	Trois personnes distinctes en production
Le cas « deuxième signature puis exécution » est prouvé par les tests, non filmé dans l'interface	Démonstration avec plusieurs comptes MetaMask

Dans `createCampaign`, le bénéficiaire est l'adresse qui crée la campagne et non un paramètre. Ce choix évite qu'un tiers crée une campagne au nom d'une autre adresse, mais diffère de la signature citée dans le sujet.

11 Perspectives

- Brûler les CFD lors d'un remboursement pour lier durablement le pouvoir de vote à l'engagement financier.
- Ajouter un plafond cumulé par période au multisig, et trois gardiens distincts en production.
- Ajouter un type de proposition de retrait pour que la gouvernance couvre aussi le retrait normal.
- Placer un petit serveur intermédiaire entre l'interface et Pinata pour ne plus exposer la clé.
- Piste complémentaire : un script Python de détection d'anomalies sur les événements `Contributed` (montants répétitifs, adresses en rafale, pics de contribution).

12 Conclusion

La plateforme répond aux quatre modules du sujet. Les cinq mécanismes de sécurité exigés (anti-réentrance, contrôle d'accès, circuit breaker, multisig 2 sur 3, snapshot du vote) sont implémentés, reliés à une notion du cours et prouvés par 81 tests. L'audit Slither ne remonte aucune vulnérabilité haute ou moyenne, et les contrats sont déployés et vérifiés sur Sepolia.

Difficultés rencontrées. La documentation encore limitée de Hardhat 3 ; l'enchaînement des corrections Slither, où corriger une alerte en a fait apparaître une autre ; l'interprétation du retrait conditionné par la gouvernance ; et l'intégration de deux parties développées en parallèle, facilitée par les interfaces et les mocks.

Apports. Ce projet nous a appris que la sécurité se joue dans l'ordre des instructions, dans la séparation des rôles et dans l'honnêteté sur les limites.

Annexes

- Rapports Slither : `docs/slither-report.txt`, `docs/slither-report-apres.txt`, `docs/slither-complet.txt` ; analyse : `docs/slither-analyse.md`.
- Adresses Sepolia : `docs/sepolia-addresses.json` ; décisions de conception : `docs/decisions.md`.
- Dépôt Git : `/contracts`, `/test`, `/frontend`, `/scripts`, `/docs` (GitHub : latouti-hassnae/plateforme-de-financement-participatif-d-centralis-e---gouvernance-par-jeton).